

Example Product JD에 맞춘 프론트엔드 포트폴리오



Sample Candidate

Frontend Engineer

sample@example.com

[github](#)

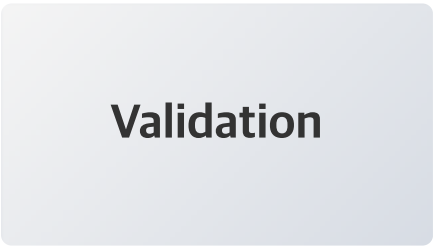
[portfolio](#)

프로젝트 목록



Sample Dashboard

- 2p 프로젝트 소개, 담당 역할, 상태 설계 방향
- 3p 핵심 기술 이슈 - TanStack Query + URL query



Sample Company Form Validation

- 4p 문제, 결정, 액션, 결과



프로젝트 소개

내부 운영자가 여러 CSV를 수동으로 합쳐 보던 주간 지표를 한 화면에서 확인하도록 만든 대시보드입니다.

담당 역할

프론트엔드 리드로 서버 상태, 필터 상태, 차트 시각화 흐름을 설계하고 구현했습니다.

- TanStack Query 기반 서버 상태 분리
- URL query 기반 필터 조건 동기화
- Recharts 기반 주요 지표와 추세 시각화

운영자 리포트 흐름 개선

1. 문제 상황

- 운영자가 여러 CSV를 수동으로 합쳐 주간 지표를 확인
- 필터 조건 공유와 같은 조건 재현이 어려움
- 리포트 작성 전 필요한 데이터를 화면마다 다시 비교해야 함

2. 과제

- 서버 데이터 상태와 사용자가 조작하는 필터 상태 분리
- 동일 조건의 화면을 URL로 공유하고 새로고침 후에도 유지
- 운영자가 지표와 추세를 한 화면에서 비교하도록 구성

3. 행동

- TanStack Query로 로딩, 재요청, 서버 상태를 관리
- 필터 조건을 URL query에 동기화해 재현 가능한 화면 상태 구현
- Recharts로 주요 지표와 추세를 시각화

4. 결과

- 주간 리포트 작성 시간이 2시간에서 30분으로 줄었다는 운영자 피드백 확보
- 필터 조건 공유와 재현이 쉬워졌다는 정성 피드백 확보

TanStack Query + URL query 상태 설계

1. 문제 상황

- 필터 변경과 데이터 요청이 한 컴포넌트 안에 섞여 변경 영향 범위가 커짐
- 운영자가 동일 조건을 공유해도 화면 상태가 재현되지 않음

2. 과제

- 서버 상태와 UI 상태를 분리해 유지보수성을 높임
- 필터 조건을 URL에 저장해 링크 공유와 새로고침을 지원

3. 행동

- 서버 데이터 요청과 캐시 관리는 TanStack Query로 이동
- 필터 조건은 URL query로 직렬화하고 화면 진입 시 복원
- 차트와 테이블은 같은 query state를 기준으로 렌더링



4. JD 연결점

- 대시보드 UI, API 연동 구조, 서버 상태 관리, 운영자 흐름 개선 요구와 직접 연결
- 업무용 화면에서 상태 설계 판단을 설명할 수 있는 사례로 사용

Sample Company Form Validation

React · TypeScript · Zod · QA

1. 문제 상황

- 폼 데이터 타입이 느슨해 런타임 오류가 QA 단계에서 반복 발견
- 사용자가 어떤 입력을 잘못했는지 화면에서 바로 알기 어려움

2. 과제

- 입력 단계에서 오류를 차단하도록 폼 스키마와 API 응답 타입 정리
- 검증 실패를 명확한 에러 메시지로 매핑

3. 행동

- Zod 스키마로 입력 데이터 구조와 검증 기준 명시
- API 응답 타입을 정리해 화면 상태와 서버 응답 간 불일치 축소
- 검증 실패 메시지를 사용자 입력 흐름에 맞게 매핑

4. 결과

- QA 재현 이슈 5건 중 4건을 사전 검증 단계에서 차단
- JD의 폼 입력 검증, TypeScript 코드 품질 개선, QA 이슈 차단 요구와 연결

